

## **Assignment 3: Text Representation & Machine Learning Classification**

*(strict action and policy regarding plagiarism and ai generated code/content – lead to no refund or certificate generation) ( use ai tools for help and write your own codes)*

### **Title**

Comparative Study of Text Vectorization Techniques and ML Classifiers for Sentiment Analysis

### **Problem Statement**

Implement multiple text representation techniques (Bag-of-Words, TF-IDF, n-grams) and train various machine learning models to classify sentiment in movie reviews. You will perform comprehensive experiments to understand how text vectorization impacts model performance and conduct feature engineering to improve classification accuracy.

### **Dataset**

#### **IMDB Movie Reviews Dataset**

- **Source:** [Stanford IMDB Dataset](#) or [Kaggle IMDB Dataset](#)
- **Description:** 50,000 movie reviews (25k positive, 25k negative)
- **Split:** 25,000 training, 25,000 testing

**Alternative:** Amazon Product Reviews

- **Source:** [UCI Amazon Reviews](#)

### **Detailed Instructions**

#### **Part 1: Data Loading and Preprocessing (10%)**

1. Load IMDB dataset (handle large files efficiently)
2. Check class distribution (balanced?)
3. Apply preprocessing pipeline from Assignment 2:
  - Text cleaning
  - Tokenization
  - Lemmatization (use either NLTK or spaCy)
  - Stopword removal (experiment with keeping sentiment words)
4. Create train/validation/test split (60/20/20)
5. Save preprocessed data for reproducibility

## **Part 2: Text Vectorization (30%)**

### **Task 2.1: Bag-of-Words (Count Vectorizer)**

1. Implement using `sklearn.feature_extraction.text.CountVectorizer`
2. Experiment with parameters:
  - `max_features`: 1000, 5000, 10000
  - `min_df`: 2, 5, 10 (minimum document frequency)
  - `max_df`: 0.8, 0.9, 1.0 (maximum document frequency)
  - `binary`: True vs False
3. Analyze vocabulary size and sparsity
4. Visualize feature distribution

### **Task 2.2: TF-IDF Vectorization**

1. Implement using `TfidfVectorizer`
2. Experiment with parameters:
  - `norm`: 'l1', 'l2', None
  - `use_idf`: True vs False
  - `smooth_idf`: True vs False
  - `sublinear_tf`: True vs False (use log scaling)
3. Compare with Bag-of-Words:
  - Top 20 features by weight
  - Distribution of TF-IDF scores
4. Visualize TF-IDF scores for sample documents

### **Task 2.3: N-gram Features**

1. Unigrams (1-gram): Individual words
2. Bigrams (2-gram): Word pairs
3. Trigrams (3-gram): Word triplets
4. Combined: (1,2), (1,3), (2,3), (1,2,3)
5. Experiments:

## Python code implementation

*# Unigrams only*

```
CountVectorizer(ngram_range=(1,1))
```

*# Unigrams + Bigrams*

```
CountVectorizer(ngram_range=(1,2))
```

*# Bigrams only*

```
CountVectorizer(ngram_range=(2,2))
```

### 6. Analyze:

- Top bigrams and trigrams
- Impact on vocabulary size
- Computational cost

## Task 2.4: Character-level N-grams

### 1. Implement character-level features:

python

```
CountVectorizer(analyzer='char', ngram_range=(2,4))
```

2. Compare with word-level features
3. Useful for handling typos and OOV words

## Part 3: Machine Learning Models (40%)

### Task 3.1: Implement 6 ML Classifiers

#### 1. Naive Bayes:

- MultinomialNB (for count data)
- BernoulliNB (for binary data)
- Tune: alpha (smoothing parameter)

#### 2. Logistic Regression:

- Tune: C (regularization strength), penalty ('l1', 'l2')
- Use class\_weight='balanced' if needed

#### 3. Support Vector Machine (SVM):

- Linear kernel: LinearSVC
- Tune: C, max\_iter

#### 4. **Random Forest:**

- Tune: n\_estimators, max\_depth, min\_samples\_split

#### 5. **Gradient Boosting** (XGBoost or LightGBM):

- Tune: learning\_rate, n\_estimators, max\_depth

#### 6. **K-Nearest Neighbors (KNN):**

- Tune: n\_neighbors, metric

### **Task 3.2: Training Pipeline**

#### **Python code implementation:**

```
from sklearn.pipeline import Pipeline
```

```
# Example pipeline
```

```
pipeline = Pipeline([
```

```
    ('vectorizer', TfidfVectorizer()),
```

```
    ('classifier', LogisticRegression())
```

```
])
```

```
pipeline.fit(X_train, y_train)
```

```
predictions = pipeline.predict(X_test)
```

### **Task 3.3: Hyperparameter Tuning**

1. Use GridSearchCV or RandomizedSearchCV
2. Tune at least 3 models thoroughly
3. Use cross-validation (5-fold)
4. Track best parameters for each model

#### **Task 3.4: Model Evaluation** For each model, calculate:

- Accuracy
- Precision, Recall, F1-score (macro and weighted)
- Confusion matrix
- ROC-AUC score
- Classification report

## Part 4: Feature Engineering (15%)

### Task 4.1: Custom Features

#### 1. Sentiment Lexicon Features:

- Count positive/negative words (use VADER or custom lexicon)
- Polarity score ratio

#### 2. Statistical Features:

- Review length (character count, word count)
- Average word length
- Sentence count
- Exclamation marks count
- Capital letters ratio

#### 3. Domain-Specific Features:

- Presence of specific keywords ("amazing", "terrible", "worst")
- Rating mentions (if available)

#### 4. Combine with Text Features:

#### Python code implementation:

```
from sklearn.preprocessing import StandardScaler
from scipy.sparse import hstack

# TF-IDF features
tfidf_features = tfidf_vectorizer.transform(X_train)

# Custom features
custom_features = StandardScaler().fit_transform(custom_df)

# Combine
combined = hstack([tfidf_features, custom_features])
```

### Task 4.2: Feature Selection

1. **Chi-square Test:** Select top k features
2. **Mutual Information:** Feature relevance
3. **L1-based Feature Selection:** Using Lasso

4. Compare model performance with/without feature selection

## Part 5: Comparative Analysis (5%)

### Task 5.1: Create Comparison Tables

#### Vectorization Model Accuracy F1-Score Training Time Inference Time

|             |     |     |     |     |     |
|-------------|-----|-----|-----|-----|-----|
| BoW (5k)    | NB  | X%  | X   | Xs  | Xms |
| TF-IDF (5k) | LR  | Y%  | Y   | Ys  | Yms |
| ...         | ... | ... | ... | ... | ... |

### Task 5.2: Analysis Questions

1. Which vectorization technique performs best?
2. Which model achieves highest accuracy?
3. What's the speed vs accuracy trade-off?
4. How does n-gram order affect performance?
5. Does feature engineering improve results significantly?

## Expected Outputs

### 1. Code Deliverables

- vectorization.py - All vectorization implementations
- models.py - ML model training scripts
- feature\_engineering.py - Custom feature extraction
- evaluation.py - Evaluation metrics and visualization
- Jupyter notebook with complete workflow

### 2. Model Files

- Saved vectorizers (pickle files)
- Trained models (pickle files)
- Best model pipeline for deployment

### 3. Visualizations (minimum 12)

- Class distribution (train/test)
- Vocabulary size vs max\_features
- Feature sparsity heatmap

- Top 20 TF-IDF features (bar chart)
- Confusion matrices (all models)
- ROC curves (all models on same plot)
- Precision-Recall curves
- Model comparison bar chart (accuracy, F1)
- Training time comparison
- Feature importance (for tree-based models)
- Learning curves (train/val accuracy vs training size)
- Error analysis: misclassified examples visualization

#### 4. **Technical Report** (5-6 pages)

- Introduction and objective
- Methodology:
  - Preprocessing approach
  - Vectorization techniques explained
  - Model descriptions
- Experiments:
  - Hyperparameter tuning process
  - Ablation studies
- Results:
  - Comprehensive comparison tables
  - Statistical significance tests (t-test)
- Error Analysis:
  - Analyze 20 misclassified examples
  - Identify common failure patterns
- Conclusion:
  - Best model recommendation
  - Limitations and future work

### Findings/Observations to Document

1. How does TF-IDF compare to Bag-of-Words in your experiments?
2. Do bigrams and trigrams improve performance? By how much?
3. Which model is most sensitive to hyperparameter tuning?
4. What types of reviews are most commonly misclassified?
5. How does vocabulary size affect model performance and training time?
6. Are there diminishing returns with larger feature sets?
7. Which custom features contribute most to improved accuracy?
8. How does class imbalance (if any) affect different models?

### Evaluation Rubric (100 points)

| Criteria              | Points | Description                                                          |
|-----------------------|--------|----------------------------------------------------------------------|
| Vectorization         | 25     | Correct implementation of BoW, TF-IDF, n-grams with parameter tuning |
| Model Training        | 30     | All 6 models trained correctly with proper hyperparameter tuning     |
| Feature Engineering   | 15     | Custom features implemented and integrated effectively               |
| Evaluation & Analysis | 15     | Comprehensive metrics, proper comparison, insightful error analysis  |
| Visualizations        | 10     | Clear, informative plots supporting analysis                         |
| Report Quality        | 5      | Well-structured, professional presentation                           |

### Bonus Points (10)

- Implement ensemble method (stacking/voting) combining top 3 models (+4)
- Achieve >90% test accuracy (+3)
- Implement LIME for model interpretability (+2)
- Cross-dataset evaluation (train on IMDB, test on Amazon) (+1)

### Target Metrics

- **Minimum:** 85% test accuracy
- **Good:** 88-90% test accuracy
- **Excellent:** >90% test accuracy